

UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

Programación II

Entrega final proyecto - Programación II

Facilitador:

Ing. Daniel Enrique Guevara Gómez

San Salvador, El Salvador

Contents

Objetivo 3

Integración de los avances anteriores 3

Arquitectura del proyecto 4

Persistencia mediante JDBC 4

Configuración de DataSource 5

Programación concurrente + base de datos 7

Cohesión y acoplamiento 8

Legibilidad y reutilización 8

Manejo de excepciones 9

Pruebas unitarias 10

Git y GitHub 11

Documentación final 11

Presentación y defensa final 13

Evidencias que deberán entregar 14

Objetivo

Integrar los conocimientos desarrollados durante el ciclo mediante la evolución de los dos avances anteriores, construyendo una aplicación Java funcional, mantenible y correctamente estructurada que incorpore:

- ✓ Programación Orientada a Objetos.
- ✓ Herencia y polimorfismo.
- ✓ Clases abstractas e interfaces.
- ✓ Colecciones y genéricos.
- ✓ Programación concurrente.
- ✓ Arquitectura por capas.
- ✓ Persistencia mediante JDBC.
- ✓ Pool de conexiones mediante DataSource.
- ✓ Cohesión y bajo acoplamiento.
- ✓ Manejo de excepciones.
- ✓ Pruebas unitarias.
- ✓ Git y GitHub.
- ✓ Buenas prácticas de desarrollo.

Integración de los avances anteriores

El proyecto final deberá incorporar y mejorar los elementos desarrollados durante los dos avances.

Debe evidenciarse:

- ✓ Encapsulamiento.
- ✓ Constructores.
- ✓ Herencia.
- ✓ Polimorfismo.
- ✓ Clases abstractas.
- ✓ Interfaces.
- ✓ Especialización de clases derivadas.
- ✓ Colecciones.
- ✓ Genéricos.
- ✓ Programación concurrente.
- ✓ Arquitectura por capas.
- ✓ Persistencia.

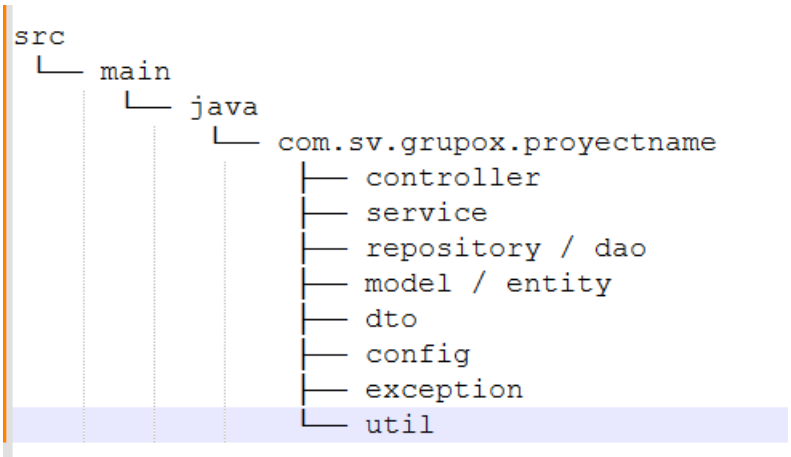
- ✓ Git/GitHub.

Las observaciones realizadas durante los avances anteriores deberán estar corregidas o justificadas.

Arquitectura del proyecto

El proyecto deberá mantener una separación adecuada de responsabilidades.

Como mínimo, se espera una estructura similar a:



Persistencia mediante JDBC

Para la entrega final, la persistencia deberá realizarse mediante una base de datos relacional utilizando JDBC.

Deben implementar como mínimo:

- ✓ Conexión a la base de datos.

- ✓ DataSource.
- ✓ Connection.
- ✓ PreparedStatement.
- ✓ ResultSet.
- ✓ executeQuery().
- ✓ executeUpdate().
- ✓ Inserción.
- ✓ Consulta.
- ✓ Actualización y/o eliminación.
- ✓ Manejo de errores de conexión.
- ✓ Cierre adecuado de recursos.

Se recomienda utilizar try-with-resources para administrar correctamente los recursos JDBC.

Configuración de DataSource

El proyecto deberá contar con una clase de configuración, por ejemplo:

DataSourceConfig

La configuración deberá demostrar el uso y comprensión de un pool de conexiones.

Se deberán configurar y documentar los siguientes parámetros:

Parámetro	Propósito
validationQuery	Consulta utilizada para validar una conexión
minIdle	Número mínimo de conexiones inactivas
maxActive	Número máximo de conexiones activas
minEvictableIdleTimeMillis	Tiempo mínimo de inactividad antes de considerar una conexión para expulsión
removeAbandonedTimeout	Tiempo para considerar abandonada una conexión

maxWait	Tiempo máximo de espera para obtener una conexión
initialSize	Número inicial de conexiones del pool
testOnBorrow	Validar la conexión al solicitarla
testOnReturn	Validar la conexión al devolverla
testWhileIdle	Validar conexiones mientras permanecen inactivas
testOnConnect	Validar la conexión al establecerla
timeBetweenEvictionRunsMillis	Intervalo entre ejecuciones de revisión de conexiones
removeAbandoned	Permitir detectar/remover conexiones abandonadas
maxIdle	Máximo de conexiones inactivas
validationInterval	Intervalo mínimo entre validaciones de una misma conexión

Ejemplo

```
@Bean
public DataSource dataSource() {
    DataSource dataSource = new DataSource();
    dataSource.setUrl(url);
    dataSource.setUsername(username);
    dataSource.setPassword(password);
    dataSource.setValidationQuery("SELECT 1");
    dataSource.setInitialSize(5);
    dataSource.setMinIdle(5);
    dataSource.setMaxActive(20);
    dataSource.setMaxIdle(10);
    dataSource.setMaxWait(10000);
   dataSource.setTestOnBorrow(true);
    dataSource.setTestOnReturn(false);
    dataSource.setTestWhileIdle(true);
    dataSource.setTestOnConnect(true);
dataSource.setTimeBetweenEvictionRunsMillis(30000);
    dataSource.setMinEvictableIdleTimeMillis(60000);
    dataSource.setRemoveAbandoned(true);
dataSource.setRemoveAbandonedTimeout(60);
    dataSource.setValidationInterval(30000);
    return dataSource;}
```

Nota: Los valores son únicamente un ejemplo. Cada equipo deberá justificar los valores seleccionados de acuerdo con las características de su aplicación.

Importante

No se evaluará solamente que los atributos aparezcan escritos en DataSourceConfig.

Programación concurrente + base de datos

La programación concurrente debe estar integrada con una operación real del sistema y debe interactuar con la base de datos.

No será suficiente realizar:

```
new Thread(() -> {
    System.out.println("Procesando...");
}).start();
```

La concurrencia deberá tener una finalidad dentro del proyecto.

Deben demostrar:

- ✓ Thread, Runnable o ExecutorService.
- ✓ Ejecución de tareas concurrentes.
- ✓ Acceso a la BD desde las tareas concurrentes.
- ✓ Uso del DataSource.
- ✓ Manejo adecuado de conexiones.
- ✓ Manejo de excepciones.
- ✓ Control de posibles problemas de concurrencia.

Cohesión y acoplamiento

Alta cohesión

Las clases deben tener responsabilidades claramente relacionadas.

Bajo acoplamiento

Las clases no deben depender innecesariamente de implementaciones concretas.

Los estudiantes deberán identificar en su documentación:

- ✓ Una clase con buena cohesión.
- ✓ Una dependencia que hayan reducido.
- ✓ Una decisión de diseño que permita disminuir el acoplamiento.

Legibilidad y reutilización

Deberán demostrar que realizaron una revisión del código para mejorar:

- ➢ Nombres de clases.
- ➢ Nombres de métodos.
- ➢ Responsabilidades.

- Reutilización.
- Separación de responsabilidades.
- Eliminación de código duplicado.
- Organización de paquetes.
- Métodos demasiado grandes.
- Clases con demasiadas responsabilidades.

Deberán incluir al menos un ejemplo de mejora antes/después.

Manejo de excepciones

El proyecto deberá implementar un manejo adecuado de errores.

Se deberá demostrar el uso apropiado de:

- try
- catch
- finally, cuando corresponda.
- throw
- throws
- Excepciones propias.
- Excepciones relacionadas con JDBC.
- Errores de negocio.

No se considerará correcto un manejo como:

```
catch (Exception e) {
    System.out.println("Error");
}
```

sin una razón técnica para hacerlo.

Pruebas unitarias

El proyecto deberá incorporar pruebas automatizadas.

Se deberán utilizar herramientas como:

- JUnit
- Mockito, cuando sea necesario aislar dependencias.

Se deberán probar, como mínimo:

- Métodos de Service.
- Métodos de Controller.
- Clases de utilidad.
- Validaciones.
- Reglas de negocio.
- Casos exitosos.
- Casos de error.

Las pruebas no deben limitarse a ejecutar métodos sin verificar resultados.

Deben utilizar assertions apropiadas:

```
assertEquals(...)
assertTrue(...)
assertFalse(...)
assertNotNull(...)
assertThrows(...)
```

Cuando exista una dependencia externa, podrán utilizar Mockito:

```
@Mock
private ClienteRepository repository;
```

Git y GitHub

El proyecto deberá mantenerse en un repositorio Git.

Se deberá evidenciar:

- Repositorio GitHub.
- Participación de todos los integrantes.
- Commits reales.
- Evolución del proyecto.
- Código actualizado.
- Documento final actualizado.

Los commits deberán representar trabajo real.

Si el repositorio no es público, no se evaluará y tendrán CERO como nota del avance.

Se espera utilizar mensajes descriptivos, por ejemplo:

- ✓ feat: implementa registro de clientes
- ✓ feat: agrega conexion JDBC
- ✓ test: agrega pruebas de ClienteService
- ✓ fix: corrige manejo de excepcion
- ✓ refactor: separa responsabilidades de VentaService

Documentación final

El documento deberá contener como mínimo:

1. Portada
2. Introducción
3. Descripción del problema
4. Objetivos

5. Requerimientos

6. Arquitectura

7. Diagrama UML actualizado

8. POO aplicada

- ✓ Encapsulamiento.
- ✓ Herencia.
- ✓ Polimorfismo.
- ✓ Abstracción.
- ✓ Interfaces.

9. Colecciones y genéricos

10. Programación concurrente

Explicar:

- ✓ Qué proceso ejecutan concurrentemente.
- ✓ Por qué utilizan concurrencia.
- ✓ Qué mecanismo utilizaron.
- ✓ Cómo interactúa con la BD.

11. JDBC

Explicar:

- ✓ Conexión.
- ✓ DataSource.
- ✓ Pool.
- ✓ DAO/Repository.
- ✓ PreparedStatement.
- ✓ ResultSet.

12. Configuración del DataSource

Documentar los parámetros solicitados.

13. Cohesión y acoplamiento

14. Legibilidad y reutilización

15. Manejo de excepciones

16. Pruebas unitarias

Incluir evidencias de ejecución.

17. Git/GitHub

18. Conclusiones

Presentación y defensa final

Cada equipo tendrá 5 minutos para responder 5 preguntas relacionadas con su propio proyecto.

Reglas

- ✓ Todos los integrantes del equipo deben estar presentes.
- ✓ Se realizarán 5 preguntas por equipo.
- ✓ Las preguntas serán seleccionadas por el docente.
- ✓ Las preguntas estarán relacionadas con los contenidos de las semanas 13, 14, 15 y 16.
- ✓ Para cada pregunta se seleccionará aleatoriamente(o no) un integrante del equipo.
- ✓ El tiempo total para las 5 preguntas será de 5 minutos.

Esto significa que todos deben conocer el proyecto, no solamente la parte que desarrollaron.

La defensa es individual, aunque el proyecto sea grupal. El hecho de que un integrante haya desarrollado una funcionalidad específica no lo exime de conocer las demás partes del proyecto. Las preguntas podrán abordar cualquier componente de la solución.

La ausencia de un integrante durante la defensa afectará la evaluación individual correspondiente a la defensa.

Evidencias que deberán entregar

La carpeta/repositorio final debería contener algo similar a:

```
/proyecto-final
│
├── src/
├── test/
├── database/
│   ├── schema.sql
│   └── data.sql
│
├── docs/
│   ├── documento-final.pdf
│   ├── uml.png
│   └── arquitectura.png
│
├── README.md
├── pom.xml
└── ...
```

El README.md debería explicar:

- ✓ Nombre del proyecto.
- ✓ Integrantes.
- ✓ Tecnologías.
- ✓ Requisitos para ejecutar.
- ✓ Configuración de BD.
- ✓ Cómo ejecutar.
- ✓ Cómo ejecutar las pruebas.
- ✓ Usuario/credenciales de prueba, si aplica.
- ✓ Descripción de la concurrencia.
- ✓ URL del repositorio.

Nota: Si para ejecutar el proyecto se necesitan pasos específicos, crear un script en bash para poder ejecutarlo y si ya lo tienen integrado con Docker, especificar la forma de ejecución. Si existe login colocar usuarios de prueba para poder acceder al sistema y probarlo.

Criterio	Puntos	Nota
Integración y evolución de los avances 1 y 2	0.5	
POO: encapsulamiento, herencia y polimorfismo	0.8	
Clases abstractas e interfaces	0.5	
Colecciones y genéricos	0.5	
Arquitectura y separación de responsabilidades	0.8	
JDBC y operaciones CRUD	1.0	
DataSource y configuración del pool	0.8	
Concurrencia integrada con BD	1.0	
Cohesión y bajo acoplamiento	0.5	
Legibilidad y reutilización	0.4	
Manejo de excepciones	0.5	
Pruebas unitarias JUnit/Mockito	0.5	
Git/GitHub	0.2	
Subtotal	**8**	

Defensa-Preguntas

Cada pregunta vale 0.4

Desempeño	Puntos	Nota
Respuesta correcta, clara y aplicada al proyecto	4	
Respuesta parcialmente correcta	2–3	
Respuesta superficial o con errores importantes	1	
No responde / respuesta incorrecta	0	

Componente	Puntos	Nota final
Proyecto	8	
Defensa	2	
TOTAL	**10**	